

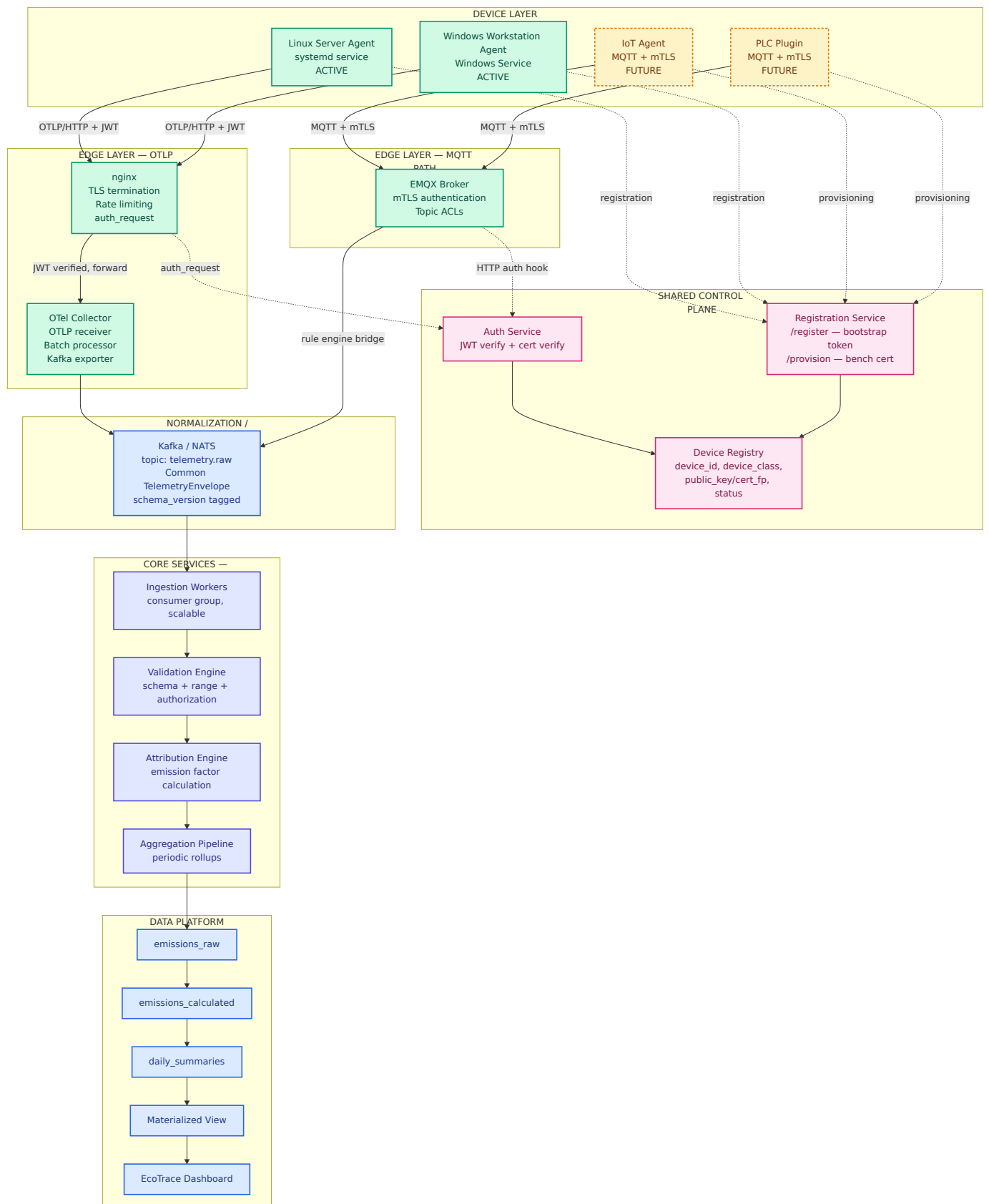
EcoTrace — Finalized Architecture & Shared Agent Core

Document purpose: This extends the finalized platform architecture with a detailed specification of the **Shared Agent Core** — the common library layer that sits on top of every general-purpose-OS collector (Linux, Windows, and any future OS-capable device class), plus the **Schema Contract** layer that keeps constrained devices (IoT, PLC) consistent without requiring shared compiled code.

Table of Contents

1. [Platform Architecture](#)
 2. [The Sharing Problem](#)
 3. [Shared Agent Core — Overview](#)
 4. [Shared Agent Core — Component Detail](#)
 5. [Ports — The Enforcement Mechanism](#)
 6. [Platform Adapters](#)
 7. [Tier 2 — Schema Contract for Constrained Devices](#)
 8. [Repository Structure](#)
 9. [Workflows](#)
 10. [Design Rules — What Must Never Be Violated](#)
-

1. Platform Architecture



This is the platform-level view: device classes at the edges, two protocol-specific edges converging on one streaming layer, and Core Services that never branch on device type. The remainder of this document zooms into the **Device Layer** — specifically, what makes the Linux and Windows agent boxes above actually maintainable as the device class count grows.

2. The Sharing Problem

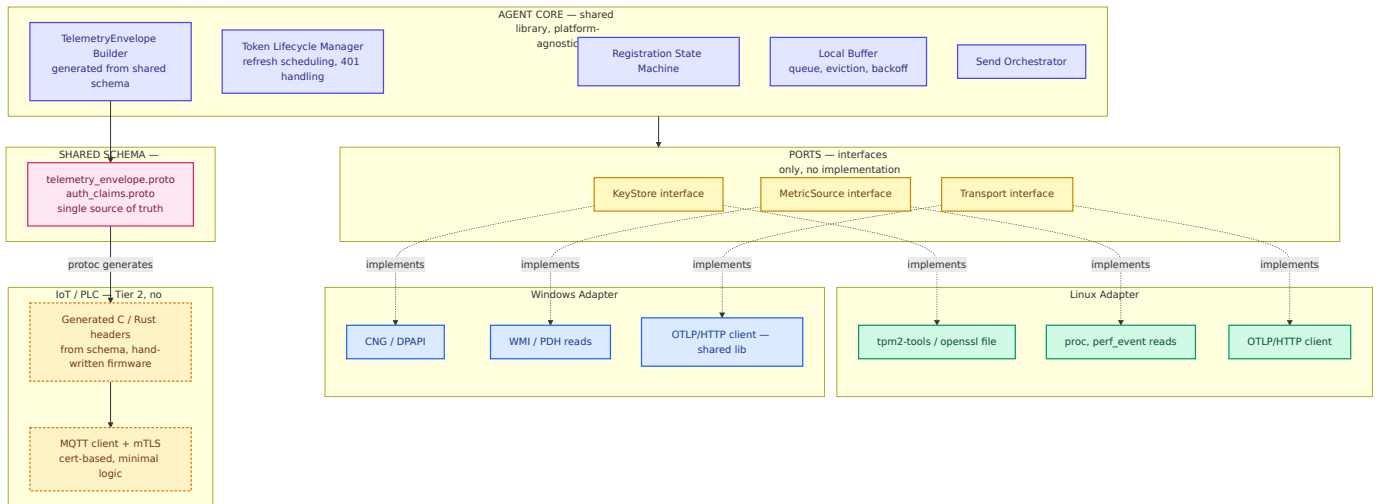
Every agent — regardless of platform — needs to perform the same five pieces of business logic:

Responsibility	Why It Must Behave Identically Everywhere
Registration flow	A device that registers differently on Linux vs Windows produces inconsistent Device Registry rows and breaks the "one Auth Service, two edges" guarantee
Token lifecycle (refresh, 401 handling)	Divergent refresh timing or retry logic across platforms creates inconsistent load patterns on the Auth Service and inconsistent failure behavior in the field
TelemetryEnvelope construction	If each platform hand-builds this JSON/protobuf independently, field names and types will silently drift apart over time
Local buffering, eviction, backoff	Inconsistent buffering means inconsistent data-loss behavior under network failure — a property that must be uniform across the fleet for the platform's reliability story to hold
Send orchestration	When to flush, how to batch, how to back off — must be one policy, not N independently-tuned policies

Implementing this five times — once per platform, by hand — is the structural risk: it **compiles**, it **looks** consistent at launch, and it silently diverges with every subsequent change because nothing enforces that a fix made in the Linux agent also lands in the Windows agent.

The fix is not "be more disciplined." The fix is to make divergence **impossible to compile**, not just unlikely.

3. Shared Agent Core — Overview



The governing principle: the Agent Core has zero knowledge of any operating system. It is pure business logic — state machines, scheduling, validation, queueing — written once. It depends only on three abstract interfaces (**ports**). Each platform supplies a concrete implementation of those interfaces (**adapters**). The compiler refuses to build a platform target unless its adapter fully satisfies every port — this is what turns "please keep these consistent" from a process rule into a build-time guarantee.

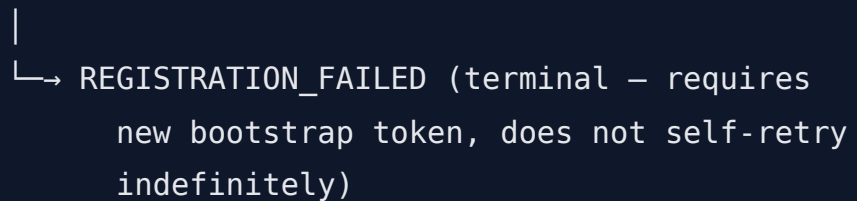
4. Shared Agent Core — Component Detail

4.1 Registration State Machine

Owns the entire lifecycle from "freshly installed, no identity" to "registered and operating."

States:

UNREGISTERED → REGISTERING → REGISTERED → OPERATING



Responsibilities:

- On startup, check local storage (via KeyStore port) for existing credentials. If present and valid → skip directly to OPERATING.
- If absent, generate a keypair via `KeyStore.GeneratePrivateKey()` and call the Registration Service's `/register` endpoint.
- Parse the response, persist `device_id` + tokens via `KeyStore.SaveCredentials()`.
- On registration failure (expired/consumed bootstrap token), transition to `REGISTRATION_FAILED` and emit a clear local log entry – this is a human-actionable failure, not a transient one to retry through.

This logic is identical in shape for Linux and Windows. The only platform-specific calls are the two KeyStore port methods — everything else (state transitions, the HTTP call to Registration Service, response parsing) lives entirely in the Core.

4.2 Token Lifecycle Manager

Responsibilities:

- Track `issued_at` and `expires_at` for the current `access_token`.
- Schedule a refresh attempt at a fixed margin before expiry (e.g. at 45 minutes into a 60-minute token lifetime).
- On a 401 response from any send attempt (not just the scheduled refresh), immediately trigger an out-of-cycle refresh rather than waiting for the next scheduled window.
- On refresh failure (refresh_token itself rejected – the platform's revocation path), transition the Registration State Machine back to `REGISTRATION_FAILED`. The agent does not attempt to silently re-register with a stale or absent bootstrap token.

The refresh margin, the 401-triggers-immediate-refresh rule, and the revocation-triggers-halt rule are platform-wide policy decisions. Encoding them once in the Core means changing the margin from 45 to 50 minutes is a one-line change that takes effect on every platform simultaneously on next release — not a change that has to be remembered and separately applied N times.

4.3 TelemetryEnvelope Builder

Responsibilities:

- Accept raw metric readings from `MetricSource.Collect()` and assemble them into the canonical envelope shape.
- Stamp `schema_version`, `device_id`, `device_class`, and the timestamp window automatically — adapters never set these fields themselves, eliminating an entire class of "adapter forgot to set a required field" bugs.
- Validate the assembled envelope against the schema before it is ever handed to the buffer, catching malformed data at the source rather than letting Validation Engine reject it downstream after a wasted network round trip.

Generated from: `schema/telemetry_envelope.proto`

(see §7 — this struct is not hand-written; it is generated so the Linux/Windows agent's definition of the envelope can never drift from the definition used by constrained-device firmware on the MQTT path.)

4.4 Local Buffer

Responsibilities:

- Queue constructed envelopes pending successful transmission.
- Enforce a maximum buffer size (time-bounded, e.g. 24 hours of windows) with a defined eviction policy (oldest-first) when the cap is exceeded.
- Persist the buffer across agent restarts (via a platform-neutral serialization the Core owns – the Core decides HOW buffered data is structured; the KeyStore/filesystem access needed to persist it is the only adapter-dependent piece).

This is the component most likely to silently diverge if left unshared – eviction policy and buffer size are exactly the kind of detail that gets "tuned" independently per platform during debugging and never reconciled. Centralizing it removes that failure mode entirely.

4.5 Send Orchestrator

Responsibilities:

- Decide WHEN to flush the buffer (on a timer, on buffer-size threshold, or immediately for high-priority events).
- Batch multiple envelopes into a single `Transport.Send()` call where the transport supports it, reducing connection overhead.
- Apply exponential backoff on send failure, with a defined maximum retry count before the data is left in the buffer for the next scheduled attempt rather than retried in a tight loop.
- Attach the current `access_token` to outgoing requests by reading it from the Token Lifecycle Manager – orchestration logic never touches credential storage directly.

5. Ports — The Enforcement Mechanism

Ports are interface definitions with **zero implementation**. They exist solely to draw a hard boundary between "logic that must be identical everywhere" and "logic that is necessarily platform-specific."

```
// core/ports.go – lives entirely inside the platform-agnostic Core

type KeyStore interface {
    GeneratePrivateKey() (publicKey []byte, err error)
    Sign(data []byte) (signature []byte, err error)
    LoadStoredCredentials() (Credentials, error)
    SaveCredentials(Credentials) error
}

type MetricSource interface {
    Collect(window TimeRange) ([]Metric, error)
}

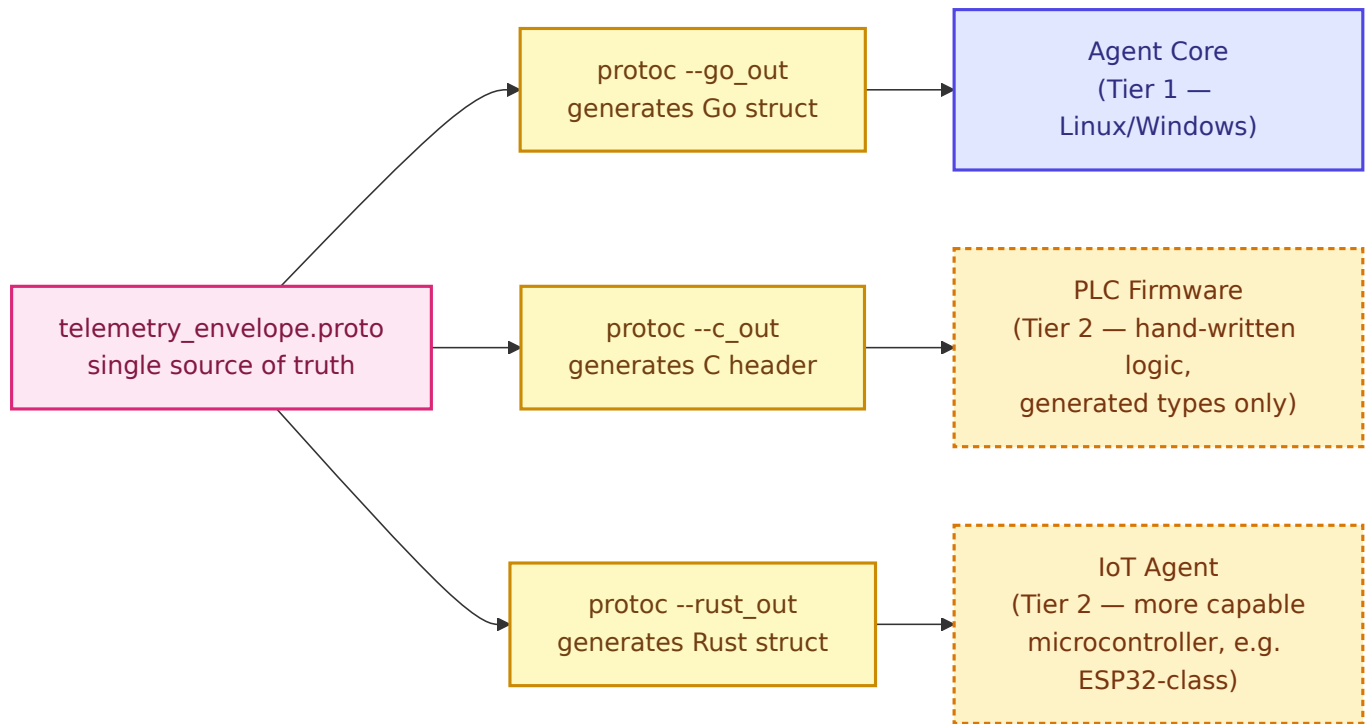
type Transport interface {
    Send(envelope TelemetryEnvelope) error
}
```

Why this is the actual fix, not just documentation: if a future contributor adds a Windows-specific registry read directly inside `core/registration.go`, the Core package would need to import a Windows-only library — which fails to compile when cross-compiling the Linux target. The boundary is structural, enforced by the Go toolchain itself, not by a code review catching it after the fact.

6. Platform Adapters

Adapters are intentionally thin. Their only job is translating a port method call into the real OS API call.

Port Method	Linux Adapter	Windows Adapter
<code>KeyStore.GeneratePrivateKey()</code>	<code>tpm2_create</code> if TPM present, else <code>openssl genpkey</code> + file at <code>0600</code>	<code>NCryptCreatePersistedKey</code> (TPM-backed CNG), else DPAPI-encrypted blob



Neither the PLC firmware engineer nor the IoT firmware engineer hand-types field names into their own struct definitions. Both generate from the same `.proto` file the Go Core also generates from. This is what prevents the slow schema drift that would otherwise occur between a team building firmware in C and a team building the Go agent, without requiring the firmware to link against the Go runtime — which it physically cannot do.

The complexity budget difference is deliberate: Tier 2 devices authenticate via a pre-burned mTLS certificate (the TLS library does almost all the work) rather than implementing the Tier 1 registration/refresh state machine, because that state machine's complexity is a reasonable ask of a Linux server and an unreasonable ask of a microcontroller.

8. Repository Structure

together at compile time, per

G00S

`go build` (Linux target) compiles `core/` + `adapter/linux/`. `G00S=windows go build` compiles `core/` + `adapter/windows/`. The same `core/` package ships unmodified in both binaries.

9. Workflows

9.1 Adding a New Tier 1 Platform (e.g. macOS, or a general-purpose edge gateway)

1. Confirm the target can run a Go binary and has SOME form of OS-level secure key storage (Keychain on macOS, for example).
2. Create `/adapter/macos/` implementing `KeyStore` and `MetricSource`. Transport is likely free – reuse `transport/otlp` directly.
3. No changes to `/core`. No changes to `/schema` unless this platform genuinely needs a new `metric_type` the schema doesn't yet define (in which case the `.proto` file gains a field, and ALL consumers – Core, any Tier 2 generated headers – regenerate from it together, never independently).
4. Build with `G00S=darwin`, ship.

9.2 Adding a New Tier 2 Device Class (e.g. a new sensor type)

1. Determine the `metric_types` this device needs to report. If they already exist in `telemetry_envelope.proto`, no schema change needed – proceed to step 3.
2. If new `metric_types` are needed, add them to the `.proto` file. Regenerate C/Rust headers via `protoc`. This is a schema PR, reviewed once, consumed by every Tier 2 firmware target.
3. Firmware engineer writes minimal device-specific logic using the generated header – reading sensor registers, populating the generated struct, publishing over MQTT with mTLS.
4. Bench-provision the device (cert issuance via Registration Service's `/provision` endpoint, per the platform-level registration workflow).
5. Validation Engine gets a new `device_class` allow-list entry (a data/config change, not a code change) defining which `metric_types` this class is authorized to report.

Neither workflow touches Core Services (Ingestion, Validation engine code, Attribution, Aggregation) as defined in the platform architecture in § 1 — both terminate in either a new adapter package or a new generated-header consumer, exactly as the platform-level design rule requires.

10. Design Rules — What Must Never Be Violated

Rule 1. The `core/` package may never import a platform-specific library, directly or transitively. If it would need to, the missing capability belongs behind a new or existing port, not inline in Core.

Rule 2. No adapter may contain business logic — state transitions, retry policy, scheduling decisions. An adapter's only job is translating a port method into a real OS or hardware call. If an adapter starts making decisions, that logic has leaked out of Core and will diverge across platforms.

Rule 3. No Tier 1 or Tier 2 target may hand-define the `TelemetryEnvelope` shape independently. Every consumer generates from `schema/telemetry_envelope.proto`. A field added on one side without updating the schema first is a process violation, not a valid shortcut.

Rule 4. A new device class never introduces a new path into Core Services (Ingestion, Validation, Attribution, Aggregation). It either gets a Tier 1 adapter or a Tier 2 generated-header integration — both terminate at the same `telemetry.raw` streaming topic, in the same envelope shape, as established in the platform architecture.

These four rules are the mechanism that makes the claim "core services never change when a new device class is added" something the architecture actually enforces — not just a goal stated in a diagram.